# Sprawozdanie na pierwsze kolokwium

Autorzy: Kacper Smykla, Dmytro Chubyk

## Opis algorytmu

Przygotowany algorytm genetyczny do optymalizacji wielokryterialnej, działają w **reprezentacji rzeczywistej**, wykonując każdy z pojedynczych eksperymentów (15 łącznie) dzieła wg następującej listy kroków:

0. Ustaw licznik zużycia wybranej funkcji F(x) na zero, zarezerwuj przestrzeń w pamięci na historię uruchomień, zainicjalizuj generatory liczb losowych
1. **Inicjalizuj populacje losowymi (jednorodnie) numerami z dziedziny** dla każdego wymiaru
2. Oceń populacje początkową za pomocą wybranej funkcji ewaluacji wielokryterialnej F(x) oraz oceń przynależność do frontów oraz dystansu crowding.
3. Dopóki liczba iteracji nie osiągnie zamierzone 500:
    a. Zarezerwuj przestrzeń na nową populację
    b. Wybierz 2 rodziców za pomocą **selekcji turniejowej**
    c. Stwórz nowego potomka na podstawie **krzyżowania** – BLX-$\alpha$
    d. Dokonaj **mutacji Cauchy** potomka ze slajdu „Mutation #1 - real valued representation", stosowanie **strategii odbijania lustrzanego** na krawędzi dziedziny
    e. Oceń kryteria danego potomka za pomocą wybranej funkcji F(x) i sprawdź czy nowe rozwiązanie poprawia wynik względem rodziców, inaczej inkrementuj licznik stagnacji
    f. Dokonaj **sukcesji elitarnej** w **strategii ($\mu + \lambda$),** algorytm próbuje zmieścić na podstawie miary crowdingu poprzednią populacje (rodzice + dzieci) do nowej populacji tak aby nie wystawała poza limit populacji.

## Parametry każdego eksperymentu:

- „ftype"- wartość typu wyliczeniowego wybranej funkcji (ZDT1,ZDT2,ZDT3,ZDT4,ZDT6)

- „domain_min" – minimalna wartość osiągalna dla prawie całej dziedziny

- „domain_max"- maksymalna wartość osiągalna dla prawie całej dziedziny

- „domain_min_x1" – minimalna wartość osiągalna w dziedzinie dla pierwszego punktu x1

- „domain_max_x1"- maksymalna wartość osiągalna w dziedzinie dla pierwszego punktu x1

- „N" – liczba wymiarów przestrzeni wartości decyzyjnych

- „sigma" – początkowa wartość współczynnika mutacji

# Stałe dla każdego eksperymentu:

- „NUM_OBJECTIVES” (2) – liczba optymalizowanych kryteriów, rozmiar przestrzeni objective (celu)

- „MAX_ITERATIONS” (500) – liczba uruchomień głównej pętli programu

- „POPULATION_SIZE” (100) – liczba osobników w populacji

- „STAGNATION_MULTIPLIER” (1.2) – mnożnik wartości sigma odpowiadającej sile mutacji, w przypadku gdy dochodzi do stagnacji wartości

- „MAX_SHIFT” (4.0) – maksymalną wartość o jaką można przesunąć gen podczas mutacji, z uwagi na rosnącą wartość siły mutacji (sigma) w przypadku stagnacji

# Wartości parametrów dla każdego z eksperymentów:

## Eksperyment 1:

- Funkcja: ZDT1

- wymiar (N): 10

- dziedzina: [0,1]

- dziedzina dla x1: [0,1]

- początkowy wsp. mutacji: 0.3

## Eksperyment 2:

- Funkcja: ZDT1

- wymiar (N): 30

- dziedzina: [0,1]

- dziedzina dla x1: [0,1]

- początkowy wsp. mutacji: 0.3

## Eksperyment 3:

- Funkcja: ZDT1

- wymiar (N): 50

- dziedzina: [0,1]

- dziedzina dla x1: [0,1]

- początkowy wsp. mutacji: 0.4

## Eksperyment 4:

- Funkcja: ZDT2

- wymiar (N): 10

- dziedzina: [0,1]

- dziedzina dla x1: [0,1]

- początkowy wsp. mutacji: 0.3

## Eksperyment 5:

- Funkcja: ZDT2

- wymiar (N): 30

- dziedzina: [0,1]

- dziedzina dla x1: [0,1]

- początkowy wsp. mutacji: 0.3

## Eksperyment 6:

- Funkcja: ZDT2

- wymiar (N): 50

- dziedzina: [0,1]

- dziedzina dla x1: [0,1]

- początkowy wsp. mutacji: 0.4

## Eksperyment 7:

- Funkcja: ZDT3
- wymiar (N): 10
- dziedzina: [0,1]
- dziedzina dla x1: [0,1]
- początkowy wsp. mutacji: 0.3

## Eksperyment 8:

- Funkcja: ZDT3
- wymiar (N): 30
- dziedzina: [0,1]
- dziedzina dla x1: [0,1]
- początkowy wsp. mutacji: 0.3

## Eksperyment 9:

- Funkcja: ZDT3
- wymiar (N): 50
- dziedzina: [0,1]
- dziedzina dla x1: [0,1]
- początkowy wsp. mutacji: 0.4

## Eksperyment 10:

- Funkcja: ZDT4
- wymiar (N): 10
- dziedzina: [0,1]
- dziedzina dla x1: [-5,5]
- początkowy wsp. mutacji: 0.5

## Eksperyment 11:

- Funkcja: ZDT4
- wymiar (N): 30
- dziedzina: [0,1]
- dziedzina dla x1: [-5,5]
- początkowy wsp. mutacji: 0.5

## Eksperyment 12:

- Funkcja: ZDT4
- wymiar (N): 50
- dziedzina: [0,1]
- dziedzina dla x1: [-5,5]
- początkowy wsp. mutacji: 0.6

## Eksperyment 13:

- Funkcja: ZDT6
- wymiar (N): 10
- dziedzina: [0,1]
- dziedzina dla x1: [0,1]
- początkowy wsp. mutacji: 0.3

## Eksperyment 14:

- Funkcja: ZDT6
- wymiar (N): 30
- dziedzina: [0,1]
- dziedzina dla x1: [0,1]
- początkowy wsp. mutacji: 0.3

## Eksperyment 15:

- Funkcja: ZDT6

- wymiar (N): 50

- dziedzina: [0,1]

- dziedzina dla x1: [0,1]

- początkowy wsp. mutacji: 0.4

# Metodologia pomiarowa i wynikowe wykresy krzywych wielokryterialnych:

Program został uruchomiony raz, gdzie dla każdego z 15 przypadków testowych uruchomiono pętle algorytmu genetycznego dla 500 obrotów pętli. Otrzymane punkty z pierwszego frontu (niezdominowane) zapisano do poszczególnych plików csv wraz z informacją z jakiej iteracji został dany punkt otrzymany. Na podstawie tego zagregowano wynik 15 plików CSV w jeden arkusz excela oraz wykonano wykres zależności wartości kryteriów od siebie dla każdego z 15 przypadków testowych, gdzie każda iteracja została przedstawiona jako oddzielna seria danych.

# Obserwacje:

Funkcja ZDT4 i ZDT6 ze względu na swoją konstrukcję początkowo stanowiła wyzwania dla mojego wielokryterialnego algorytmu genetycznego. O ile dla funkcji ZDT1-3 bardzo szybko wraz z upływem iteracji algorytmu zacząłem otrzymywać oczekiwaną krzywe F2 od F1, tak pozostałe funkcje sprawiały problem, nie zależnie od wybranego wymiaru. Udało mi się to rozwiązać poprzez zaimplementowanie lepszych operatorów krzyżowania – BLX-α (względem poprzedniego krzyżowania arytmetycznego, które nadal nieużywane pozostało w kodzie do podmiany) oraz operatora mutacji Cauchy, który powoduje pewne większe skoki w przestrzeni co pozwala wyjść z lokalnych optimów. Także bardzo pomogło doimplementowanie licznika stagnacji, gdy kolejne iteracje dzieci nie poprawiają wyników oraz aplikowanie potęgowo współczynnika stagnacji, który przemnaża małą wartość siły mutacji (sigma) przez swój współczynnik stagnacji. Dzięki temu otrzymałem lepsze wyniki dla funkcji ZDT4 i ZDT6. Wymagało to jednak poczytania na temat tych rozwiązań w Internecie, co spowolniło moje prace nad algorytmem.